

Week 1 Assignment: Python Fundamentals for Data Analytics

Task – 1:

Source Code:-

```
print("Welcome to Student Information Program!")

name = input("Enter your Name: ")

college = input("Enter your College Name: ")

branch = input("Enter your Branch: ")

print("\n----- Student Information -----")

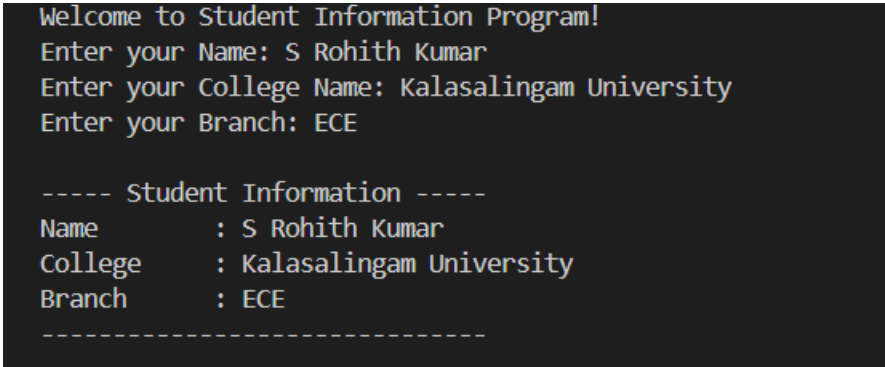
print("Name      :", name)

print("College   :", college)

print("Branch    :", branch)

print("-----")
```

Output Screenshot:-

A screenshot of a terminal window with a dark background. It shows the output of a Python program. The first part shows the program's welcome message and three input prompts. The user has entered 'S Rohith Kumar' for the name, 'Kalasalingam University' for the college, and 'ECE' for the branch. The second part shows the formatted output of this information, with labels and values aligned to the right, separated by colons. The output is enclosed in a dashed border.

```
Welcome to Student Information Program!
Enter your Name: S Rohith Kumar
Enter your College Name: Kalasalingam University
Enter your Branch: ECE

----- Student Information -----
Name       : S Rohith Kumar
College    : Kalasalingam University
Branch     : ECE
-----
```

Explanation:-

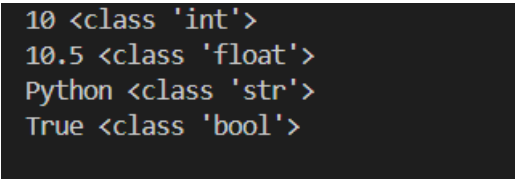
The program shows a welcoming message to the user, prompts for the name, college name, and branch of the user, and then outputs the data entered in a clear way.

Task – 2:

Source Code:-

```
integer = 10  
  
float_value = 10.5  
  
string = "Python"  
  
boolean = True  
  
print(integer, type(integer))  
  
print(float_value, type(float_value))  
  
print(string, type(string))  
  
print(boolean, type(boolean))
```

Output Screenshot:-



```
10 <class 'int'>  
10.5 <class 'float'>  
Python <class 'str'>  
True <class 'bool'>
```

Explanation:-

This program illustrates various data types in Python:

- integer stores an integer (10).
- float_value stores a float (10.5).
- string stores a string ("Python").
- boolean stores a boolean (True).
- type() determines the data type of each variable.
- print() prints the value and data type of each variable.

Task – 3:

Source Code:-

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
operation = input("Enter the operation (+, -, *, /, %): ")

if operation == "+":
    result = num1 + num2
elif operation == "-":
    result = num1 - num2
elif operation == "*":
    result = num1 * num2
elif operation == "/":
    if num2 != 0:
        result = num1 / num2
    else:
        result = "Division by zero is not allowed"
elif operation == "%":
    if num2 != 0:
        result = num1 % num2
    else:
        result = "Modulus by zero is not allowed"
else:
    result = "Invalid operation"

print("Result:", result)
```

Output Screenshot:-

```
Enter the first number: 23
Enter the second number: 21
Enter the operation (+, -, *, /, %): -
Result: 2.0
```

Explanation:-

This program takes two inputs from the user and then asks the user which type of arithmetic operation he/she wants to execute. After that, the program executes the specified arithmetic operation using the operator selected by the user such as (+, -, *, /, %).

Task – 4:-

Source Code:-

```
marks = float(input("Enter your marks: "))
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "Fail"
print("Grade:", grade)
```

Output Screenshot:-

```
Enter your marks: 98
Grade: A
```

Explanation:-

Marks for the student are fed into this program as inputs and grades are decided based on if, elif, and else statements. Grades of A are given when the marks are above or equal

to 90, B when between 75 to 89, C when marks range from 60 to 74, and F when less than 60.

Task – 5:-

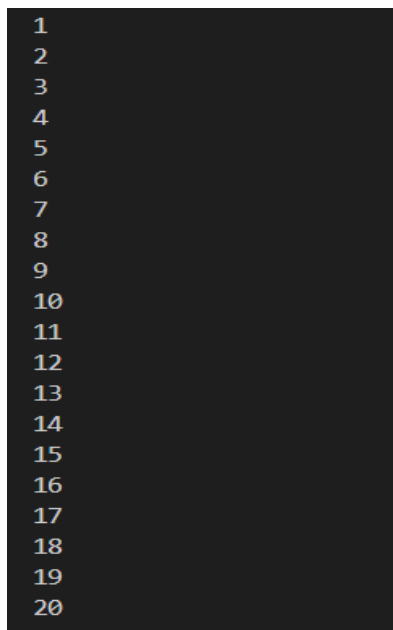
Q1. Print numbers from 1 to 20

Source Code:-

```
for i in range(1, 21):
```

```
    print(i)
```

Output Screenshot:-



```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Q2. Multiplication table of any number

Source Code:-

```
num = int(input("Enter a number: "))
```

```
for i in range(1, 11):
```

```
    print(num, "x", i, "=", num * i)
```

Output Screenshot:-

```
Enter a number: 6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
```

Q3. Print even numbers from 1 to 50 using a while loop

Source Code:-

```
i = 2
while i <= 50:
    print(i)
    i += 2
```

Output Screenshot:-

```
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
```

Explanation:-

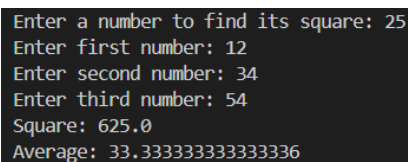
- Program one is using for loop with range() function to print numbers from 1 to 20.
- Program two will take a number from the user and then it will print the multiplication table of the entered number from 1 to 10 using for loop.
- Third program is using while loop that starts from 2 and incrementing by 2 each time and prints even numbers up to 50.

Task – 6:-

Source Code:-

```
def square(number):  
    return number * number  
  
def average(num1, num2, num3):  
    return (num1 + num2 + num3) / 3  
  
number = float(input("Enter a number to find its square: "))  
  
num1 = float(input("Enter first number: "))  
num2 = float(input("Enter second number: "))  
num3 = float(input("Enter third number: "))  
  
print("Square:", square(number))  
  
print("Average:", average(num1, num2, num3))
```

Output Screenshots:-

A screenshot of a terminal window showing the output of the program. The text is as follows:
Enter a number to find its square: 25
Enter first number: 12
Enter second number: 34
Enter third number: 54
Square: 625.0
Average: 33.333333333333336

Explanation:-

These two user defined functions are described in the following program:

square(): This function calculates the square of a number.

average(): This function finds the average of three numbers.

The program receives input from the user and uses both functions to get the result.

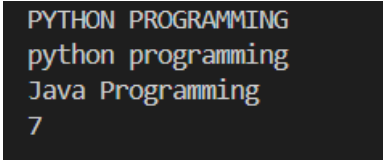
Task – 7:

Q1. String Operations

Source code:-

```
text = "Python Programming"
print(text.upper())
print(text.lower())
print(text.replace("Python", "Java"))
print(text.find("Programming"))
```

Output Screenshot:-

A screenshot of a terminal window with a dark background. It displays the output of the Python code: 'PYTHON PROGRAMMING' (all caps), 'python programming' (all lowercase), 'Java Programming' (first letter of each word capitalized), and '7' (the index of the first occurrence of 'Programming').

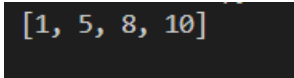
```
PYTHON PROGRAMMING
python programming
Java Programming
7
```

Q2. List Operations

Source Code:-

```
numbers = [5, 2, 8, 1]
numbers.append(10)
numbers.remove(2)
numbers.sort()
print(numbers)
```

Output Screenshot:-

A screenshot of a terminal window with a dark background. It displays the output of the Python code: '[1, 5, 8, 10]', which is the sorted list after removing the value 2 and appending 10.

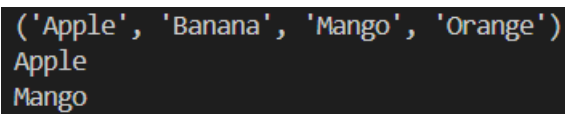
```
[1, 5, 8, 10]
```

Q3. Tuple Creation and Indexing

Source Code:-

```
fruits = ("Apple", "Banana", "Mango", "Orange")  
  
print(fruits)  
  
print(fruits[0])  
  
print(fruits[2])
```

Output Screenshot:-



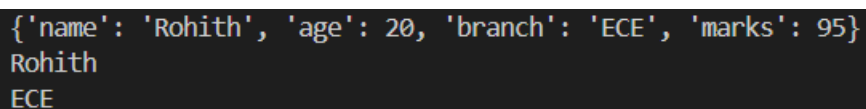
```
('Apple', 'Banana', 'Mango', 'Orange')  
Apple  
Mango
```

Q4. Dictionary Storing Student Information

Source Code:-

```
student = {  
    "name": "Rohith",  
    "age": 20,  
    "branch": "ECE",  
    "marks": 95  
}  
  
print(student)  
  
print(student["name"])  
  
print(student["branch"])
```

Output Screenshot:-



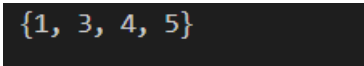
```
{'name': 'Rohith', 'age': 20, 'branch': 'ECE', 'marks': 95}  
Rohith  
ECE
```

Q5. Set Operations

Source Code:-

```
numbers = {1, 2, 3, 4}
numbers.add(5)
numbers.remove(2)
print(numbers)
```

Output Screenshot:-



```
{1, 3, 4, 5}
```

Explanation:-

- String: upper() function makes the string in uppercase, lower() makes it in lowercase, replace() replaces the string and find() finds the position of the string.
- List: append() adds the item, remove() removes the item and sort() sorts the items in ascending order.
- Tuple: A tuple can store more than one value, and indexing is done to access the individual items.
- Dictionary: Store student details as key-value pairs like name, age, branch, marks.
- Set: The add() method adds the element to the set, and the remove() method removes an existing element from the set.

Task – 8:

Source code:-

```
introduction = "Hello, my name is Rohith. I am a student of ECE and I am learning Python programming."
```

```
with open("introduction.txt", "w") as file:
```

```
    file.write(introduction)
```

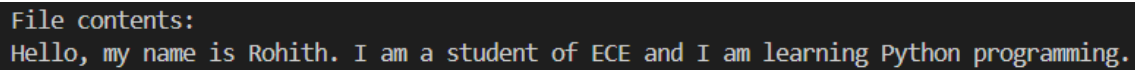
```
with open("introduction.txt", "r") as file:
```

```
    content = file.read()
```

```
print("File contents:")
```

```
print(content)
```

Output Screenshot:-



```
File contents:  
Hello, my name is Rohith. I am a student of ECE and I am learning Python programming.
```

Explanation:-

The program generates a text file titled “introduction.txt” with an introduction written in it. After that, the program opens the file in read mode, uses read() to read what is in the file, and prints it to the screen.

Task- 9:

Source code:-

```
students = []
```

```
while True:
```

```
    print("\nStudent Record Management System")
```

```
    print("1. Add Student")
```

```
    print("2. Display All Students")
```

```
    print("3. Search Student")
```

```
    print("4. Delete Student")
```

```
    print("5. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":
```

```
    name = input("Enter student name: ")
```

```
    age = input("Enter student age: ")
```

```
    branch = input("Enter student branch: ")
```

```
    marks = input("Enter student marks: ")
```

```
student = {  
    "name": name,  
    "age": age,  
    "branch": branch,  
    "marks": marks  
}  
students.append(student)  
print("Student added successfully.")
```

```
elif choice == "2":  
    if len(students) == 0:  
        print("No student records found.")  
    else:  
        for student in students:  
            print("Name:", student["name"])  
            print("Age:", student["age"])  
            print("Branch:", student["branch"])  
            print("Marks:", student["marks"])  
            print()
```

```
elif choice == "3":  
    name = input("Enter student name to search: ")  
    found = False  
    for student in students:  
        if student["name"].lower() == name.lower():  
            print("Name:", student["name"])  
            print("Age:", student["age"])  
            print("Branch:", student["branch"])
```

```
print("Marks:", student["marks"])
```

```
found = True
```

```
break
```

```
if not found:
```

```
    print("Student not found.")
```

```
elif choice == "4":
```

```
    name = input("Enter student name to delete: ")
```

```
    found = False
```

```
    for student in students:
```

```
        if student["name"].lower() == name.lower():
```

```
            students.remove(student)
```

```
            print("Student deleted successfully.")
```

```
            found = True
```

```
            break
```

```
if not found:
```

```
    print("Student not found.")
```

```
elif choice == "5":
```

```
    print("Program ended.")
```

```
    break
```

```
else:
```

```
    print("Invalid choice.")
```

Output Screenshot:-

Adding Students:-

```
Student Record Management System
1. Add Student
2. Display All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter student name: Rohith
Enter student age: 21
Enter student branch: ECE
Enter student marks: 96
Student added successfully.

Student Record Management System
1. Add Student
2. Display All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter student name: kumar
Enter student age: 21
Enter student branch: ECE
Enter student marks: 91
Student added successfully.
```

Displaying students:-

```
Student Record Management System
1. Add Student
2. Display All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 2
Name: Rohith
Age: 21
Branch: ECE
Marks: 96

Name: kumar
Age: 21
Branch: ECE
Marks: 91
```

Ending the program:-

```
Student Record Management System
1. Add Student
2. Display All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 5
Program ended.
```

Explanation:-

- The program employs a list of dictionaries to hold records of students.
- Add Student: It adds details of students to the list.
- Display All Students: It displays all records of students.
- Search Student: It searches for a student with their names.
- Delete Student: It deletes records of a particular student using their name.
- Exit: It stops the operation of the program.
- It uses a while loop to repeat the menu until “Exit” is selected by the user.

Drive Link for the python files of the tasks:

https://drive.google.com/drive/folders/1H0Hie1jHYrvAZJPIRy79eptNoglOdXXr?usp=drive_link